

Supporting Industrial Test-Failure Analysis with LLM-Based Systems: An Experience Report

Eric Jansson¹, Per Strandberg²^[0000–0003–1688–6937], Thomas Sörensen²,
Eduard Paul Enoiu¹^[0000–0003–2416–4205], and Wasif Afzal¹^[0000–0003–0611–2655]

¹ Mälardalen University, Västerås, Sweden eric95.jansson@gmail.com,
{eduard.paul.enoiu,wasif.afzal}@mdu.se

² Westermo Network Technologies AB, Västerås, Sweden
{per.strandberg,thomas.sorensen}@westermo.com
<https://www.westermo.com>

Abstract. This study examines tool-augmented Large Language Model (LLM) systems for supporting Root Cause Analysis (RCA) of nightly test failures at Westermo Network Technologies AB. Nightly test executions produce heterogeneous test data and logs that practitioners currently inspect manually across multiple sources. We implemented an RCA workflow in single-agent and orchestrated multi-agent configurations, both with access to test metadata and logs. An exploratory industrial case study used two real failure scenarios. Six practitioners evaluated the scenario reports through a survey and focus group, and operational measurements were collected from 120 repeated executions. The evaluation covered practitioner-perceived correctness, reasoning quality, fix realism, clarity, usefulness, and trust, as well as cost, duration, and consistency. Neither configuration showed a consistent practitioner-perceived quality advantage across the two scenarios. The single agent system generated reports faster and at lower cost, making it the more practical baseline in this context. The potential benefits of agent architectures require further evaluation in more complex scenarios.

1 Introduction

Logs are used to understand system behavior and diagnose failures [27]. As systems scale, log volume and diversity increase, making analysis harder. In this paper, multi-source test data denotes test metadata and log evidence retrieved from the controller and multiple devices under test. The sources may differ in schema, message format and content.

Automated log analysis can reduce manual effort but often relies on fixed patterns or log formats, limiting its use with varied or domain-specific logs [27]. LLMs can process both textual and semi-structured data, including system logs [18]. For complex tasks requiring information retrieval and multi-step analysis, they can also be combined with external tools and structured reasoning [19].

Root cause analysis (RCA) of test failures aims to determine why a test failed and what should be investigated next. In industrial testing, this often requires

engineers to examine detailed execution logs and connect events across the test environment [9]. RCA involves detecting abnormal events and interpreting these in relation to the test context and expected system behavior. Because failures may originate from misconfiguration, timeouts, aborted tests, and interactions between system components, several sources of evidence may be required. Logs can show what happened, while monitoring data, topology information, and expert knowledge may be needed to explain why it happened [30].

LLM-based agents can retrieve external data and use it in iterative test log analysis, where evidence may be distributed across several devices [15]. A single agent handles the full task, while a multi-agent system divides it among specialized agents. Even if this may improve separation of responsibilities, it also adds coordination overhead that could affect report clarity, usefulness, trust, cost and execution time. Evidence in using both configurations on the same industrial RCA task remains limited.

Industrial Context: At Westermo Network Technologies AB (hereafter, Westermo), nightly regression tests exercise network devices through an internal test framework. When a test fails, practitioners inspect test meta-data, identify the devices involved and search controller and device logs to determine if the failure originates in product software, the test framework, the test environment or their interaction. Relevant evidence may be distributed across several devices and time windows, and the current workflow therefore relies heavily on practitioner experience.

This paper reports industrial experience that examines which of the two concrete configurations provides the more appropriate baseline for the current task and what changes would be needed before integration into routine test-failure analysis at Westermo. Once given a failure case, both configurations autonomously retrieve and analyze test data and logs and generate an RCA report for practitioner assessment and further investigation. Using an exploratory industrial case study and mixed-methods design, the architectures were evaluated through system measurements, scenario-based surveys and a focus group. The evaluation considered cost, duration, consistency and practitioners' perceptions of the usefulness and trustworthiness of the generated RCA reports.

The paper contributes: (1) a tool-augmented RCA workflow instantiated in single-agent and orchestrated multi-agent configurations; (2) an industrial evaluation of these configurations using the same model, data sources, tools and report format based on two real failure scenarios, feedback from six practitioners and 120 repeated executions; and (3) practitioner-grounded lessons concerning evidence presentation, workflow integration, uncertainty and the operational trade-offs of agent specialization. This paper is based upon the Master's thesis work of the first author [10].

2 Background

Automated log analysis often transforms unstructured messages into structured event templates. However, template-based methods can be sensitive to changing formats, heterogeneous components, and minor wording differences, limiting their use in dynamic industrial environments [27].

LLMs can process natural language and semi-structured data, including system logs, making them suitable for test-failure analysis [18]. However, effective analysis requires domain-specific context that may not be available in the model’s training data, such as test metadata, device mappings, logs, topology information, and technical documentation. This information can be supplied through external tools or retrieval mechanisms [14]. Complex test-log analysis may therefore require several interactions in which the system retrieves evidence, interprets the available context, and generates a structured RCA report [19].

3 Related Work

Prior log-analysis methods have used structured classification, semantic representations, and fine-tuned language models for fault diagnosis and anomaly detection [1, 29, 31]. Other approaches combine multiple evidence sources, such as logs, performance indicators, and topology information, to support root cause analysis [30]. LLM-based methods have also been applied to log interpretation and fault localization [17, 21]. However, these studies mainly detect abnormal events or localize faults, whereas this study evaluates complete RCA reports intended to help practitioners assess evidence and continue an investigation.

Recent work has used LLMs and tool-augmented agents to support RCA more directly. RCACopilot collects diagnostic information before generating root-cause predictions and explanations [5], while ReAct-style agents and RCA-agent retrieve evidence dynamically from external diagnostic services [19, 25]. Structured and multi-agent approaches, including Flow-of-Action and MA-RCA, divide diagnosis into controlled steps or specialized responsibilities [8, 16]. Most of these systems are evaluated in cloud or operational incident settings rather than industrial software testing, and they provide limited evidence about how practitioners assess the resulting reports.

Research outside RCA also suggests that architectural complexity is not automatically beneficial. SWE-agent shows that agent performance depends strongly on the design of tool interfaces and feedback [28], while AGENTLESS demonstrates that predefined workflows can remain competitive with more autonomous agent designs [26]. These findings support treating architecture as a task-dependent design choice. Prior work provides limited comparisons between single-agent and multi-agent configurations on the same industrial RCA task, using the same data, tools, and report format. This study addresses that gap through practitioner evaluation and operational measurements of cost, duration, and semantic output stability.

Together, these works suggest that architecture should be treated as a design choice, where added specialization and context separation may help but also introduce coordination overhead and additional failure points. Overall, prior work demonstrates the value of tool use, workflow control and agent specialization for RCA. However, most studies concern cloud operations, microservices and incident triage. This motivates the examination of these approaches in industrial embedded system software testing, where the available evidence primarily consists of test metadata, device mappings, and logs.

4 Study Design and Evaluation Process

This method section provides an overview of the study design, including the system development process and an industrial case study at Westermo.

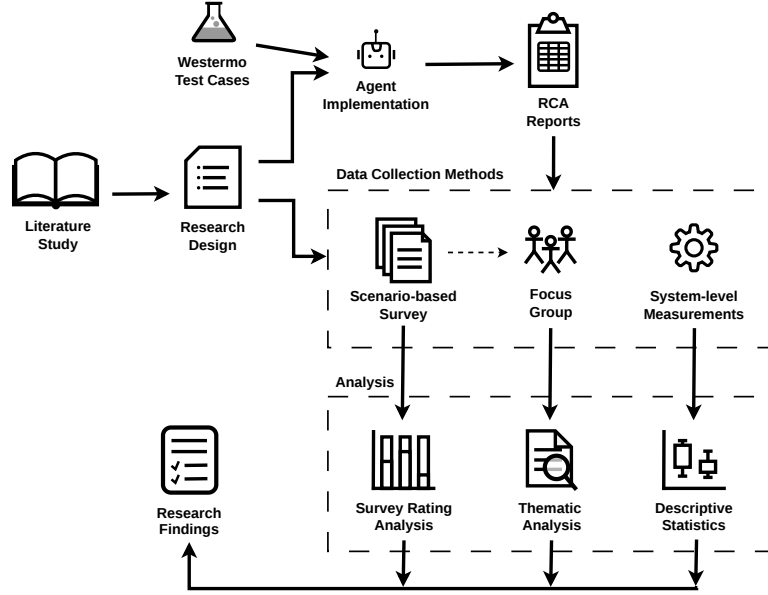


Fig. 1: Overview of the study design and evaluation process.

We conducted an exploratory industrial case study at Westermo using mixed-methods data collection [4, 20, 22]. As illustrated in Figure 1, the literature review informed the system design, evaluation metrics and data collection methods. The implementation was constrained by Westermo’s existing infrastructure, data formats and available test-failure scenarios. The evaluation dimensions were informed by previous work distinguishing correctness from explanation quality [2],

using inference time as an efficiency measure [6] and relating useful explanations to failure reasoning and plausible corrective actions [11].

Four real test failures were selected in consultation with test framework experts at Westermo. Two failures were used during development to refine the prompts, tool instructions, and retrieval behavior. The remaining two were reserved for evaluation and used in the practitioner survey and focus group. For each evaluation scenario, both configurations received the same initial failure information and had access to the same external data sources.

Six Westermo practitioners evaluated reports generated for the two evaluation scenarios through a survey followed by a semi-structured focus group. The participants represented test framework architecture and development, operating system development, release management, agile coordination and project management. The focus group lasted approximately 80 minutes and was conducted in a hybrid format, with five participants attending in person and one remotely. The survey assessed six dimensions: perceived correctness, reasoning quality, realism of the proposed fix, clarity, usefulness and trust. The focus group explored the reports’ reasoning, the useful and misleading guidance, the level of detail, the structure, and the perceived differences between the single and multi-agent systems. With participant consent, the discussion was recorded and transcribed. The first author manually coded relevant segments using the predefined themes and identified recurring observations and differences between participants. Representative quotations were selected to explain patterns in the survey results.

System measurements were collected independently from the practitioner evaluation. Both architectures were executed 30 times for each of the two scenarios, producing 120 RCA reports in total. Three metrics were recorded. *Cost* was calculated from Azure’s *CostByResource* data using input, cached-input and output-token usage obtained from execution traces. *Duration* measured the time from initiating an analysis until the final report was produced. *Consistency* represented the semantic stability of reports generated repeatedly from the same input. An LLM evaluator compared the *Root Cause* and *Evidence* sections and grouped reports with semantically similar conclusions. Consistency was calculated as the size of the largest similarity group, divided by the number of 30 reports generated for that scenario and architecture.

Survey ratings and system measurements were analyzed descriptively due to the small participant group and the exploratory study design. The qualitative findings were used to interpret the rating distributions and explain how practitioners assessed the reports in relation to their work.

5 An Implementation of LLM-based Agents

This section describes two LLM-based systems for supporting RCA of nightly test failures at Westermo. The single-agent variant handled the entire workflow, while the multi-agent variant divided tasks between an orchestrator and specialized subagents. Both used the same data sources, tool categories, and report structure to enable a controlled comparison.

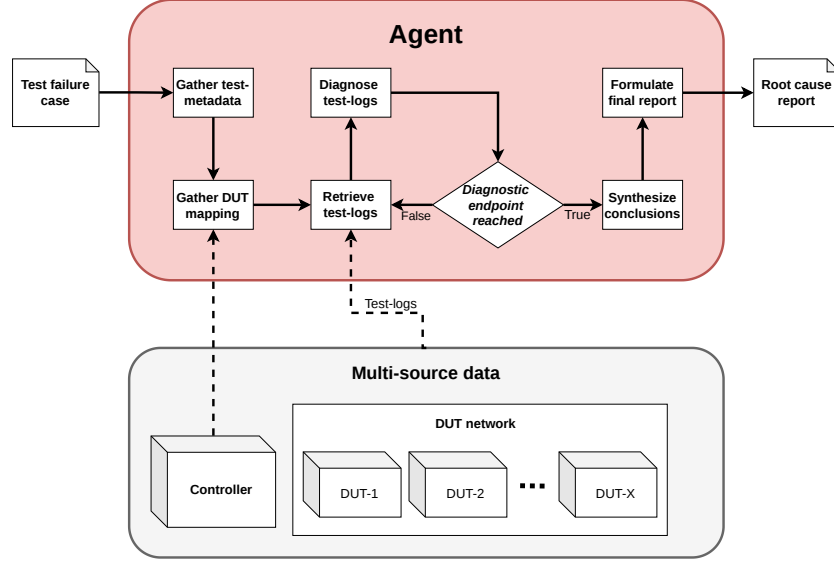


Fig. 2: RCA workflow and multi-source test data used by the agent systems.

5.1 Agent Workflow

Figure 2 presents the workflow used by both agent systems. Starting from a failed test case, the agents retrieve test metadata, identify the relevant devices under test (DUTs), collect logs, analyze the evidence and generate an RCA report. Evidence may originate from the controller and several devices involved in the same failure.

External test data was accessed through two Model Context Protocol (MCP) servers: one for test result metadata and another one for test logs. Only tools needed for metadata, device mapping and log retrieval were exposed. In the single agent system, the main agent called these tools directly, while the multi-agent system assigned them to specialized subagents coordinated by an orchestrator. The internal MCP interfaces and retrieval parameters are described only at a high level because they are proprietary to Westermo.

The workflow was defined through prompts rather than enforced as a fixed sequence of code. This allowed the agents to select tools dynamically while following the same general process. After retrieving metadata and identifying the relevant DUTs, the agents iteratively collected and analyzed logs until they deemed the available evidence sufficient.

The final report followed a common structure. The *Root Cause* field stated the most likely cause of the failure, while *Confidence* indicated the agent’s uncalibrated confidence in that conclusion. *Evidence* presented the most relevant supporting log entries, and *Reasoning Steps* explained how the evidence was interpreted. *Next Steps* suggested actions for confirming, investigating, or resolv-

ing the failure. Finally, *Assumptions* identified unverified conditions used in the analysis, while *Limitations* described missing information or other constraints affecting the conclusion.

5.2 Implementation Platform and Configuration

Azure AI Foundry was selected because it aligned with Westermo’s infrastructure and data protection requirements for processing internal logs, prompts, tool outputs and RCA reports. GPT-5.4 was selected because it was the model supported in Westermo’s deployment and workflow at the time of the experiment. The systems were implemented in Python using *Deep Agents*³, a LangChain-based framework supporting tool use, subagents, middleware and multi-step execution. Each system combined the Azure-hosted LLM, a restricted set of MCPs and custom tools, and prompts that define the RCA workflow.

5.3 Prompt Design and Agent Responsibilities

Both systems used modular prompts that included workflow instructions, tool descriptions, evidence-grounding rules, output requirements and domain knowledge of Westermos test logs. These modules were combined at runtime before the specific failure case was provided.

The prompt was initially developed for the single-agent system and then distributed across the multi-agent architecture. The *orchestrator* received the overall workflow and coordination instructions, while each subagent received only the context needed for its responsibility. The *metadata subagent* retrieved test information, the *mapping subagent* identified the relevant DUTs and the *log-analysis subagent* retrieved and interpreted log evidence. The *RCA-report subagent* then combined the findings into the final structured report. Formatting instructions controlled how information was returned between the subagents and the orchestrator. The structure of prompts is discussed in section 6.6 of [10].

5.4 Token Efficiency and Execution Monitoring

Reducing token usage was important because retrieved logs accounted for most of the model’s context and directly affected costs. Log entries were therefore filtered to retain only timestamps and message content, reducing the log context by approximately 80%. Query filters and retrieval limits further constrained repeated access to logs. *Langfuse* was used to trace executions, inspect tool calls, debug unexpected behavior, and collect data on duration, cost, and token usage. During development, live monitoring was also used to identify repeated tool calls and cases where agents failed to progress toward a final report.

³ Deep Agents documentation: <https://docs.langchain.com/oss/python/deepagents>

6 Results

This section reports findings from the survey, focus group and 120 system executions. Six experienced practitioners evaluated both scenarios, and each combination of scenario architectures was executed 30 times. While this section presents the summary of the results, further diagrams and details are available in the master’s thesis report [10].

6.1 Practitioner Survey Results

The survey data is treated as one source of evidence within the mixed-methods evaluation. Given the small participant group and ordinal ratings, the results are presented descriptively using rating distributions and median differences. The symbol Δ denotes the difference between the multi-agent and single-agent median ratings. Figure 3 shows a slight tendency toward positive ratings, al-

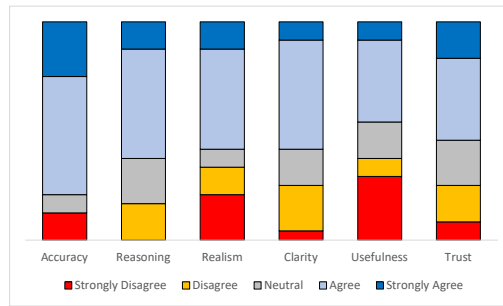


Fig. 3: Overall survey rating distribution across all evaluated reports. Each metric includes 24 ratings from six participants evaluating four report combinations.

though strongly negative responses also occurred. All metrics except *Reasoning* included both the lowest and highest ratings.

The results were scenario-dependent. The single-agent report received higher median ratings across all metrics in Scenario 1, while the multi-agent report received higher ratings in Scenario 2. The largest difference concerned *Usefulness*, with $\Delta = -1.5$ in Scenario 1 and $\Delta = +2.0$ in Scenario 2. This crossover indicates no consistent practitioner-perceived advantage for either architecture. Although the single-agent report in Scenario 1 received more positive ratings, some negative responses were included. The single-agent report in Scenario 2 had the most negative overall rating distribution among the four report combinations.

6.2 System-Level Metrics

System-level measurements were collected from 120 executions using the same scenarios as the practitioner survey. Each scenario–architecture combination was

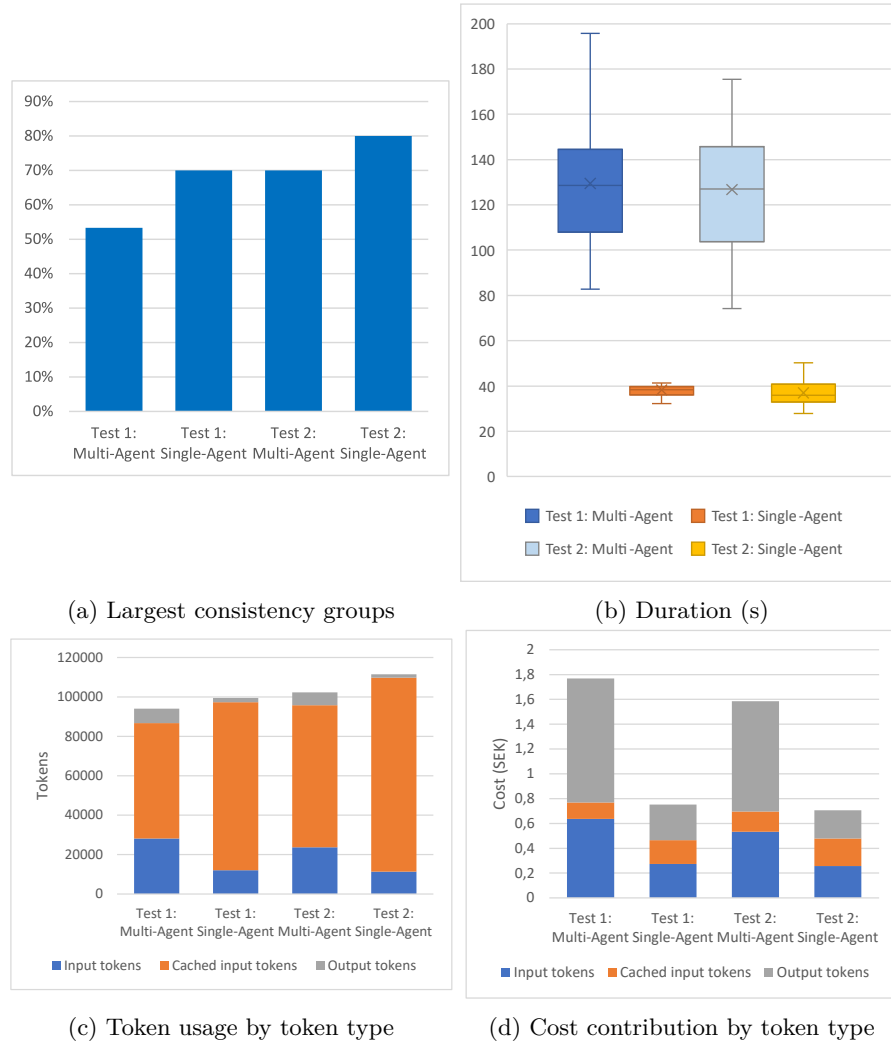


Fig. 4: Measurements gathered from 30 test analyses for each test-scenario and architecture combination. The X marks the mean; boxes show the percentiles, and the whiskers show minimum and maximum values.

executed 30 times, resulting in 60 runs per architecture. Figure 4a shows that the single-agent system produced larger consistency groups in both scenarios, with the highest value observed in Scenario 2. Figure 4 shows that the single-agent required approximately 40 seconds per report, compared with about 130 seconds for the multi-agent. It also cost approximately 0.75 SEK (0.07) per run, while the multi-agent cost 1.5–1.75 SEK (0.15) and showed greater variation in execution time. Although the single-agent used more total tokens, mainly cached

input tokens, as shown in Figure 4c, Figure 4d shows that uncached input and output tokens accounted for most of the higher multi-agent cost.

6.3 Focus Group Results

This section presents the qualitative findings from the focus group discussion. The findings are organized according to the predefined themes used in the focus group and analysis. These themes were used to structure the analysis of the transcribed discussion and to interpret how the participants understood, evaluated, and reasoned about the AI-generated RCA reports.

Theme 1: Understanding the AI’s Reasoning. Participants considered parts of the reports unnecessarily long and repetitive. The *Root Cause* section sometimes restated error messages already presented as *Evidence*, while references to tool behavior and retrieval limits were confusing and irrelevant to the failure itself. The *Reasoning Steps* section was also perceived as unnatural:

“...I probably cannot explain my own reasoning steps, so reading it feels slightly unnatural.”

In contrast, the *Next Steps* section helped participants understand how the agent interpreted the failure and what to investigate next.

Theme 2: Where the AI Provides Useful Guidance. Perceived usefulness depended on practitioners’ experience and the complexity of the failure. Reports provided little value for familiar or simple failures. However, participants considered them potentially more useful for less experienced practitioners or for unfamiliar failures. The *Evidence* section was repeatedly identified as one of the most valuable parts because it directed attention to relevant logs before the proposed cause was accepted or rejected. Participants also suggested classifying whether a failure originated from the test framework or device software. The uncalibrated *Confidence* value was difficult to interpret and participants were more likely to use the reports if they were integrated into the test framework and provided direct links to the referenced logs.

Theme 3: When the AI Leads Practitioners in the Wrong Direction. Participants warned that plausible but incorrect conclusions could direct attention toward irrelevant causes and continue influencing the investigation after the report was set aside:

“...because it [the agent] says this looks normal, then you might not even think about looking there.”

Incorrect timestamps or time windows in the *Evidence* section could also make referenced logs difficult to locate. Participants therefore expected that experience with the reports would be needed to understand their strengths and limitations.

Theme 4: Explanation Detail and Structure Participants generally preferred the most relevant information to appear first. The preferred entry point depended on the context: *Root Cause* could be useful in meetings, while *Evidence* was preferred when investigating failures in the test framework. Low or medium *Confidence* values should also be visible early.

Less relevant sections, including *Reasoning Steps*, *Assumptions*, and *Limitations*, could initially be hidden. Participants also recommended shorter root-cause descriptions, reduced timestamp precision, and time windows rather than isolated timestamps to improve readability.

Theme 5: Perceived Differences Between Architectures Participants generally perceived no systematic difference between reports from the two architectures:

“For me, you could have randomized this completely and I would not have seen any difference...”

A few participants preferred particular multi-agent reports, but this appeared to reflect their perceived correctness rather than an identifiable architectural characteristic. One multi-agent report also exposed an internal log retrieval limit that participants found confusing. This suggests that subagent behavior was not always fully hidden, although the study does not establish that this problem was unique to the multi-agent architecture.

7 Discussion

The results indicate two main findings. First, the evaluated architectures did not show a consistent difference in practitioner perceived report quality across the two scenarios. Second, the single-agent configuration provided a substantially better operational trade-off among execution time, cost, and semantic output stability. The following sections discuss these findings, their implications for tool design and industrial adoption and the main threats to validity.

7.1 Architecture and Operational Trade-offs

The practitioner ratings were primarily scenario-dependent. The single-agent report received higher median ratings across all dimensions in Scenario 1, while the multi-agent report received higher ratings in Scenario 2. The practitioner evaluation therefore provides no consistent evidence that either configuration produced better reports. This interpretation is supported by the focus group in which participants generally reported few visible differences between the outputs.

The system-level measurements showed clearer differences. The single-agent configuration generated reports approximately three times faster and at about half the cost of the multi-agent configuration. It also produced a larger share of reports reaching the same underlying root-cause conclusion in both scenarios, as shown in Figure 4a. This represents semantic output stability and not

correctness, since repeated reports may consistently reach the same incorrect conclusion.

The difference in cost was not explained by total token usage alone. Figure 4c shows that the single-agent configuration used more tokens overall, mainly because of cached input tokens. However, Figure 4d shows that the multi-agent configuration used more costly uncached input and output tokens. Its higher cost and longer execution time are consistent with the additional agent calls, intermediate outputs and coordination required between the orchestrator and subagents.

One possible explanation for the limited practitioner-perceived difference is that the evaluated scenarios and restricted tool set did not require the main potential strengths of a multi-agent design. Specialized agents may be more useful when an investigation involves a larger number of tools, substantial context or clearly separable responsibilities. In the present setting, the limited tool set reduced the need for specialization and coordination.

Context isolation also creates a trade-off. Subagents can process context-heavy tasks and return concise findings to the orchestrator, potentially reducing the amount of information handled in the main context. However, relevant details may be omitted or simplified during this transfer. The value of the multi-agent design therefore depends on whether subagent outputs preserve the necessary evidence without introducing high output-token cost or coordination overhead.

Neither configuration had access to the WeOS, test case or test framework source code and was therefore limited to diagnosing failures from logs and meta-data. Access to source code, version-control history, runtime metrics, technical documentation or issue-tracking data could support deeper analysis, while also increasing the demands on tool selection and context management. The findings indicate that, for the specific implementations, tools and scenarios evaluated in this study, the single-agent configuration provided the better operational trade-off without a consistent disadvantage in practitioner-perceived report quality.

7.2 Practitioner-Perceived Usefulness

The survey results show similar patterns for *clarity* and *trust*, as seen in Figure 3. The *usefulness* ratings varied more strongly and included the highest number of *Strongly Disagree* responses. Focus group feedback indicates that perceived correctness influenced usefulness. However, the survey was not designed to investigate this association. In addition, the ratings may additionally reflect uncertainty about how the reports would fit into existing RCA workflows.

The focus group showed that usefulness depended on participants' experience and the quality of the generated conclusion. Experienced practitioners reported less need for assistance with familiar failures, while more complex or unfamiliar cases could increase the value of the reports.

Participants identified the *Evidence* section as one of the most useful elements. Relevant log entries provided a starting point for further investigation, even when the proposed root cause was not accepted. This suggests that the

system’s practical value may lie more in evidence retrieval than in producing a definitive diagnosis.

Incorrect but plausible conclusions could also misdirect attention and increase investigation effort. Reports should therefore present conclusions as hypotheses supported by inspectable evidence, while practitioners retain responsibility for the final diagnosis.

7.3 Ethical Considerations

Because RCA reports may influence technical decisions, they should provide transparent and evidence-grounded explanations. Practitioners retain responsibility for accepting, questioning or rejecting the proposed conclusions. This is important because LLM outputs may appear plausible despite being incomplete or unsupported [7, 12, 24].

The use of internal test data also requires appropriate access controls and agreements preventing submitted information from being stored or reused [7, 13]. The practitioner evaluation followed informed consent, anonymization, restricted access, and recording deletion procedures [23]. Future deployment should also consider the computational cost and energy use of repeated LLM calls [3].

7.4 Agent Implementation Differences

Both architectures used similar tools, context and instructions to improve comparability. However, this may have limited architecture specific optimization, since single and multi-agent systems may require different prompt and context designs. This particularly affected the multi-agent system, whose prompts were adapted from the single-agent baseline. Communication between the orchestrator and subagents may therefore not have been fully optimized. The multi-agent implementation should consequently be interpreted as a controlled variant rather than an optimized architecture.

During execution, the multi-agent system also occasionally triggered Azure’s “*jailbreak*” filter or produced invalid log query syntax. These issues may reflect overlapping instructions or incomplete context transfer between agents. Overall, the observations suggest that prompts, communication formats and tool instructions should be tailored to each architecture.

7.5 Threats to validity

Each participant rated one fixed report for each scenario–configuration combination. Run-to-run variation and the procedure used to select the reports may therefore explain part of the observed results. The study also lacks an independently established ground truth and a timed manual RCA baseline. Therefore, correctness and usefulness represent practitioners’ perceptions, while system duration reflects report-generation overhead rather than a demonstrated reduction in end-to-end RCA effort. The study does not show that either configuration

is more accurate or faster than current manual practice. Participant familiarity with the test environment varied across roles, potentially influencing judgments of correctness and usefulness. Finally, our study compares one controlled implementation of each architecture rather than single-agent and multi-agent architectures in general. Cost and duration depend on the selected Azure deployment, model version, prompt-caching behavior and pricing during the experiment. The semantic-stability groupings also relied on an LLM evaluator without independent human validation.

8 Conclusions and Lessons Learned

This study evaluated LLM-based agents for supporting RCA of nightly test failures at Westermo. Neither architecture showed a consistent advantage in practitioner-perceived usefulness or report quality. Participants valued the reports mainly for locating relevant evidence and guiding further investigation.

The following lessons summarize the main implications derived from the practitioner feedback and system-level measurements.

Lessons Learned:

Start with a single agent baseline. The multi-agent configuration provided no quality advantage but was approximately three times slower and twice as costly.

Prioritize evidence over definitive diagnoses. Participants valued relevant log evidence even when they rejected the proposed root cause.

Design reports for the practitioner workflow. Detailed reasoning, repeated error descriptions, and internal tool information were considered unnecessary or confusing.

Communicate uncertainty carefully. The confidence values were difficult to interpret, and plausible but incorrect conclusions could misdirect investigations.

Together, these lessons suggest that the value of LLM-based RCA lies primarily in supporting evidence discovery and further investigation. In the evaluated setting, a simple, evidence-focused single-agent configuration therefore represents the most appropriate starting point. These lessons remain specific to the studied environment, scenarios, participants, and implementations and should be evaluated further in broader and more complex industrial settings.

Future work should also investigate RCA workflows that prioritize evidence discovery and log navigation. This includes evaluating broader agent capabilities through source-code inspection and test analysis. Further research should also examine how structured RCA outputs can support other tasks, such as pull-request creation and automated bug fixing.

Acknowledgments

This work is supported by the Swedish Agency for Innovation (Vinnova) through the project FLEXATION and by the Eureka Cluster on Software Innovation (ITEA) through the project MONA LISA.

References

1. Almodovar, C., Sabrina, F., Karimi, S., Azad, S.: LogFiT: Log Anomaly Detection Using Fine-Tuned Language Models. *IEEE TNSM* **21**(2) (2024)
2. Anghel, C., Anghel, A.A., Pecleanu, E., Craciun, M.V., Cocu, A., Niculita, C.: PEARL: A Rubric-Driven Multi-Metric Framework for LLM Evaluation. *MDPI Information* **16**(11) (2025)
3. Bender, E.M., Gebru, T., McMillan-Major, A., Shmitchell, S.: On the Dangers of Stochastic Parrots: Can Language Models Be Too Big? *FAccT'21*, ACM (2021)
4. Braun, V., Clarke, V.: Using thematic analysis in psychology. *Qualitative Research in Psychology* **3**(2) (2006)
5. Chen, Y., Xie, H., Ma, M., Kang, Y., Gao, X., Shi, L., Cao, Y., Gao, X., Fan, H., Wen, M., Zeng, J., Ghosh, S., Zhang, X., Zhang, C., Lin, Q., Rajmohan, S., Zhang, D., Xu, T.: Automatic Root Cause Analysis via Large Language Models for Cloud Incidents. In: *Proceedings of the Nineteenth European Conference on Computer Systems, EuroSys 2024, Athens, Greece, April 22-25, 2024*. pp. 674–688. ACM (2024). <https://doi.org/10.1145/3627703.3629553>
6. Cui, T., Ma, S., Chen, Z., Xiao, T., Zhao, C., Tao, S., Liu, Y., Zhang, S., Lin, D., Liu, C., Cai, Y., Meng, W., Sun, Y., Pei, D.: LogEval: A comprehensive benchmark suite for LLMs in log analysis. *Empir. Softw. Eng.* **30**(6), 173 (2025). <https://doi.org/10.1007/S10664-025-10701-6>
7. Deng, C., Duan, Y., Jin, X., Chang, H., Tian, Y., Liu, H., Wang, Y., Gao, K., Zou, H.P., Jin, Y., et al.: Deconstructing the ethics of large language models from long-standing issues to new-emerging dilemmas: A survey. *AI and Ethics* **5**(5), 4745–4771 (2025)
8. Fu, F., Ding, H., Qin, Y., Yu, J., Xu, D.: Leveraging multi-agent framework for root cause analysis. *Complex & Intelligent Systems* **12**(1) (2026). <https://doi.org/10.1007/s40747-025-02096-0>
9. Hermawan, R., Bucaioni, A., Enoiu, E., Afzal, W.: Benchmarking Large Language Models for Root Cause Analysis in Train Control Software Testing. In: *MODELS-C'25*. IEEE (2025)
10. Jansson, E.: AI-Driven Root Cause Analysis of Multi-Source Test Data. Master's thesis, Mälardalen University, Västerås, Sweden (2026), <https://www.diva-portal.org/smash/record.jsf?pid=diva2:2074203>
11. Kang, S., An, G., Yoo, S.: A Quantitative and Qualitative Evaluation of LLM-Based Explainable Fault Localization. *Proc. ACM Softw. Eng.* **1**(FSE) (Jul 2024)
12. Kazim, E., Koshiyama, A.S.: A high-level overview of AI ethics. *Patterns* **2**(9) (2021)
13. Khan, A.A., Badshah, S., Liang, P., Waseem, M., Khan, B., Ahmad, A., Fahmideh, M., Niazi, M., Akbar, M.A.: Ethics of AI: A Systematic Literature Review of Principles and Challenges. In: *EASE'22*. ACM (2022)
14. Lakatos, R., Pollner, P., Hajdu, A., Joó, T.: Investigating the Performance of Retrieval-Augmented Generation and Domain-Specific Fine-Tuning for the Development of AI-Driven Knowledge-Based Systems. *Machine Learning and Knowledge Extraction* **7**(1) (2025)
15. Masterman, T., Besen, S., Sawtell, M., Chao, A.: The Landscape of Emerging AI Agent Architectures for Reasoning, Planning, and Tool Calling: A Survey (2024), <https://arxiv.org/abs/2404.11584>
16. Pei, C., Wang, Z., Liu, F., Li, Z., Liu, Y., He, X., Kang, R., Zhang, T., Chen, J., Li, J., Xie, G., Pei, D.: Flow-of-Action: SOP Enhanced LLM-Based Multi-Agent System for Root Cause Analysis. *WWW'25*, ACM (2025)

17. Qi, J., Huang, S., Luan, Z., Yang, S., Fung, C., Yang, H., Qian, D., Shang, J., Xiao, Z., Wu, Z.: LogGPT: Exploring ChatGPT for Log-Based Anomaly Detection. In: HPCC/DSS/SmartCity/DependSys'23 (2023)
18. Raiaan, M.A.K., Mukta, M.S.H., Fatema, K., Fahad, N.M., Sakib, S., Mim, M.M.J., Ahmad, J., Ali, M.E., Azam, S.: A Review on Large Language Models: Architectures, Applications, Taxonomies, Open Issues and Challenges. *IEEE Access* **12**, 26839–26874 (2024)
19. Roy, D., Zhang, X., Bhawe, R., Bansal, C., Las-Casas, P., Fonseca, R., Rajmohan, S.: Exploring LLM-Based Agents for Root Cause Analysis. In: FSE'24. ACM (2024)
20. Runeson, P., Höst, M.: Guidelines for conducting and reporting case study research in software engineering. *Empirical software engineering* **14**(2), 131–164 (2009)
21. Shan, S., Huo, Y., Su, Y., Li, Y., Li, D., Zheng, Z.: Face It Yourselves: An LLM-Based Two-Stage Strategy to Localize Configuration Errors via Logs. In: ISSTA'24. ACM (2024)
22. Storey, M.A., Hoda, R., Maciel Paz Milani, A., Baldassarre, M.T.: Guiding principles for mixed methods research in software engineering. *Empirical Software Engineering* **30**(5), 138 (2025)
23. Strandberg, P.E.: Ethical Interviews in Software Engineering. In: ESEM'19 (2019)
24. Strandberg, P.E., Enou, E.P., Frasheri, M.: Ethical challenges and software test automation. *AI and Ethics* **5**(6), 6185–6206 (2025)
25. Wang, Z., Liu, Z., Zhang, Y., Zhong, A., Wang, J., Yin, F., Fan, L., Wu, L., Wen, Q.: RCAgent: Cloud Root Cause Analysis by Autonomous Agents with Tool-Augmented Large Language Models. In: CIKM'24. ACM
26. Xia, C.S., Deng, Y., Dunn, S., Zhang, L.: Demystifying LLM-Based Software Engineering Agents. *Proc. ACM Softw. Eng.* **2**(FSE) (2025)
27. Xue, K., Han, Q., Han, S., Shi, Z., Qiao, Y.: A Review of Software Testing Process Log Parsing and Mining. In: SSE'24. IEEE (2024)
28. Yang, J., Jimenez, C.E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., Press, O.: SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *Advances in Neural Information Processing Systems* **37** (2024)
29. Zhang, X., Xu, Y., Lin, Q., Qiao, B., Zhang, H., Dang, Y., Xie, C., Yang, X., Cheng, Q., Li, Z., Chen, J., He, X., Yao, R., Lou, J.G., Chintalapati, M., Shen, F., Zhang, D.: Robust log-based anomaly detection on unstable log data. In: ESEC/FSE'19. ACM (2019)
30. Zhang, Y., Guan, Z., Qian, H., Xu, L., Liu, H., Wen, Q., Sun, L., Jiang, J., Fan, L., Ke, M.: CloudRCA: A Root Cause Analysis Framework for Cloud Computing Platforms. In: CIKM'21. ACM (2021)
31. Zou, D.Q., Qin, H., Jin, H.: UiLog: Improving Log-Based Fault Diagnosis by Log Analysis. *Journal of Computer Science and Technology* **31**(5), 1038–1052 (2016)